

IMPLEMENTAZIONE REALISTICA DI UN PROCESSORE RISC-V

UTILIZZIAMO UN SOTTOINSIEME LIMITATO DI ISTRUZIONI:

- INTERAZIONE MEMORIA : LW e SW.
- ARITMETICO LOGICHE : ADD, SUB, AND e OR.
- SALTO CONDIZIONATO : BEQ.

RISC-V È PROGETTATO PER ESSERE **SEMPRE**, IL SET DI ISTRUZIONI SONO **REGOLARI** (I CAMPI SONO SEMPRE NELLO STESSO POSTO).

6 PASSAGGI DEL DATA PATH:

- 1) **FETCH DELL'ISTRUZIONE** : IL **PROGRAM COUNTER** FORNISCE L'INDIRIZZO DI MEMORIA DELL'ISTRUZIONE DA PRELEVARE (SALVANDOLO NELL' **INSTRUCTION REGISTER**).
- 2) **DECODIFICA** : UN **DECODER** IDENTIFICA L'OPERAZIONE DA ESEGUIRE E PREPARA LE RISORSE NECESSARIE PER L'ESECUZIONE.
- 3) **LETTURA DEI REGISTRI** : ALCUNE ISTRUZIONI RICHIEDONO 1 SOLO REGISTRO (COME **LW** e **SW**), ALTRE RICHIEDONO 2 REGISTRI (**ARITMETICO LOGICHE**).
- 4) **ESECUZIONE** : LA **ALU** ESEGUE L'OPERAZIONE SUI DATI E PRODUCE UN RISULTATO (ALTRE ISTRUZ. NON LO PRODUCONO).
- 5) **SCRITTURA DEI RISULTATI** : IL RISULTATO VIENE SALVATO NEL **REGISTRO** O NELLA **MEMORIA** (A SECONDA DELL'ISTRUZ.).
- 6) **AGGIORNAMENTO PROGRAM COUNTER** : IL PC PUÒ ESSERE AGGIORNATO IN MANIERA **CONDIZIONALE O NON**, A SECONDA DELL'ISTRUZIONE.

GLI ELEMENTI LOGICI DELL'UNITÀ DI ELABORAZIONE RISC-V SONO 2:

- 1) **ELEMENTI COMBINATORI** : L'OUTPUT DIPENDE SOLO DAGLI INPUT IN UN DATO ISTANTE.
ES: L'ALU → NON POSSIENE MEMORIA INTERNA.
- 2) **ELEMENTI DI STATO** : CONTENGONO **MEMORIA INTERNA** E MANTENGONO LO STATO TRA I CICLI DI CLOCK.
ES: MEMORIA ISTRUZ., MEMORIA DATI E REGISTRI.

LA **LOGICA SEQUENZIALE** SFRUTTA GLI ELEMENTI DI STATO

ELEMENTI SEQUENZIALI : $OUTPUT = INPUT + STATO INTERNO$

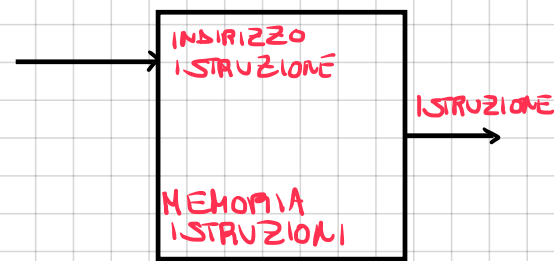
ES: FLIP-FLOP, REGISTRI, MEMORIE. → MODIFICANO IL LORO CONTENUTO IN BASE AL **CLOCK**.

- **MEMORIA ISTRUZIONI** :

È UN ELEMENTO DI STATO, IN SOLA LETTURA

CHE CONTIENE TUTTE LE ISTRUZIONI CHE IL PROCESSORE DOVRÀ ESEGUIRE DURANTE IL

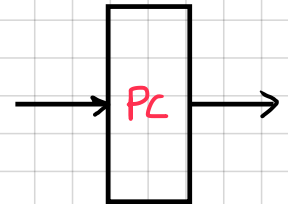
PROGRAMMA. (QUINDI LE ISTRUZIONI AL SUO INTERNO CAMBIERANNO IN BASE AL PROGRAMMA DA ESEGUIRE).



- **PROGRAM COUNTER** : È UN ELEMENTO DI STATO CHE

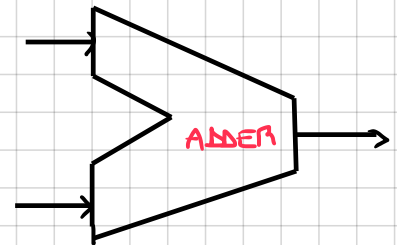
CONTIENE L'INDIRIZZO DELLA PROSSIMA ISTRUZIONE DA ESEGUIRE. È UN REGISTRO A 32 BIT CHE VIENE

AGGIORNATO AD OGNI CICLO DI CLOCK. (+4 BYTE)



- **ADDER** : È UN SOMMATORE, QUINDI UN CIRCUITO

LOGICO COMBINATORIO E NON HA MEMORIA



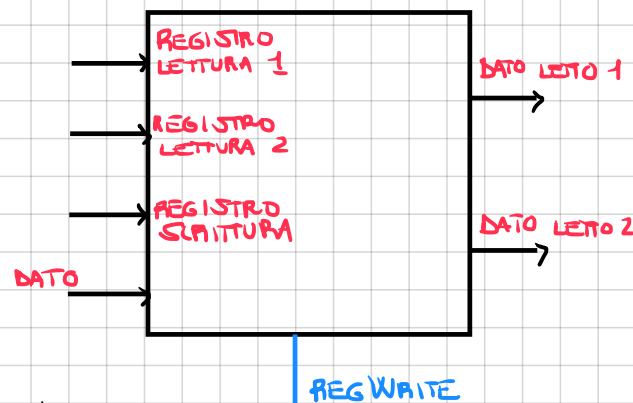
- **FILE REGISTER** : È UN ELEMENTO DI STATO E

CONTIENE 32 REGISTRI DA 32 BIT, E DISPONE DI 2 PORTE DI LETTURA E 1 DI SCRITTURA.

AD OGNI ISTANTE FORNISCE I 2 DATI DEI

2 REGISTRI IN LETTURA, E LA SCRITTURA

NEL REGISTRO DI SCRITTURA AVVIENE OGNI FRAME DI CLOCK (INOLTRE ACCETTA UN DATO DA SCRIVERE).



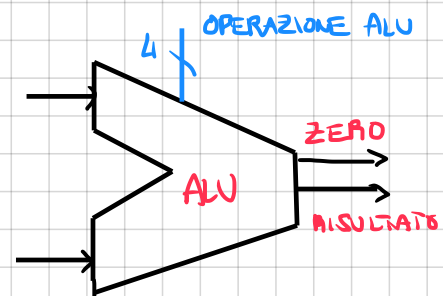
- **ALU** : È UN CIRCUITO COMBINATORIO CHE

PRENDE IN INPUT 2 VALORI DA 32 BIT E GENERARE UN OUTPUT A 32 BIT.

L'OPERAZIONE DELLA ALU DEVE ESSERE SELEZIONATA TRA LE 4 DISPONIBILI (AND, OR, ADD, SUB).

NEL CASO DI **BEQ** VIENE FATTA UNA SUB, E SE

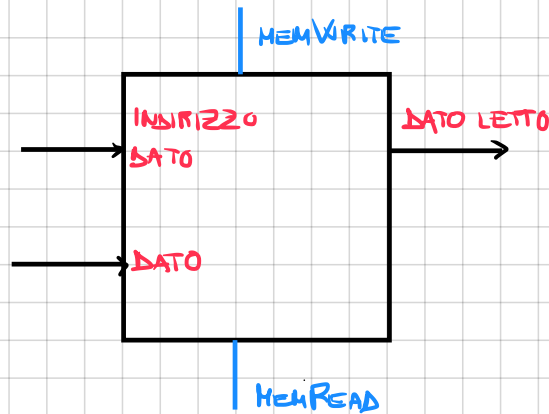
IL RISULTATO È **ZERO** ALLORA I 2 VALORI SONO UGUALI.



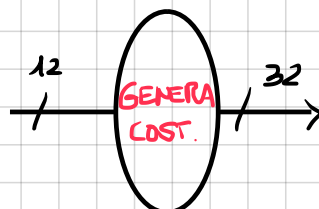
6 **MEMORIA DATI**: È UN ELEMENTO DI STATO CHE PERMETTE LETTURA E SCRITTURA.

LETTURA: RICEVE L'INDIRIZZO DEL DATO E FORNISCE IN OUTPUT IL DATO CORRISPONDENTE.

SCRITTURA: RICEVE UN INDIRIZZO E UN DATO, AGGIORNANDO LA CELLA DI MEMORIA.

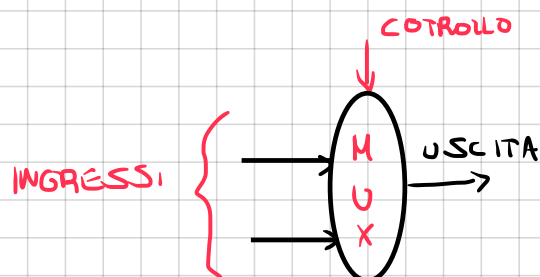


• **UNITÀ ESTENSIONE DEL SEGNO**: RICEVE 12 BIT IN INPUT (ISTRUZ. **LOAD**, **STORE** O **SALTO CONDIZIONATO**) E FORNISCE IL VALORE CON **SEGNO ESTESO A 32 BIT**.



IL DATO IN INGRESSO DI UNA ALU PUÒ ESSERE UN **REGISTRO** O UN **CAMPO IMMEDIATO**, QUINDI NON POSSIAMO COLLEGARLO DIRETTAMENTE.

QUINDI COLLEGHIAMO UN **MULTIPLEXER**:



SCEGLI IN BASE AD UN CONTROLLO SOLO 1 DEGLI N-INGRESSI.

7 SEGNALI DI CONTROLLO:

MESSO A 0

MESSO A 1

REGWRITE → NULLA

PERMETTO DI SCRIVERE IN UN REGISTRO DEL FILE REGISTER

ALUOP →

DA VEDERE DOPO

ALUSRC → L'INPUT DELLA ALU PROVIENE DAL FILE REGISTER (CONTENUTO DI REG. 2).

L'INPUT DELLA ALU PROVIENE DAL CAMPO IMMEDIATO ESTESO 32 BIT

PCSRC → $PC + 4$

$PC + \text{OFFSET}$ (CALCOLATO DAL SOMMATORE)

MEMREAD → NULLA

IL CONTENUTO DELLA MEMORIA VIENE LETTO

MEMWRITE → NULLA

IL CONTENUTO DELLA MEMORIA VIENE SCRITTO

MEMTOREG → IL DATO DA SCRIVERE NEL FILE REGISTER PROVIENE DALLA MEMORIA DATI (LOAD WORD)

IL DATO DA SCRIVERE NEL FILE REGISTER PROVIENE DALLA ALU (OPERAZIONI LOGICO ARITMETICHE)

• **UNITÀ DI CONTROLLO**: SVOLGE IL RUOLO DI **DECODER**, RICEVE L'ISTRUZIONE E IMPOSTA:

- LE LINEE DI CONTROLLO DEI **3 MULTIPLEXER**. (1 BIT)
- L'OPERAZIONE DELL'ALU (2 BIT).
- SEGNALI DI **LETTURA / SCRITTURA** NELLA MEMORIA DATI E FILE REGISTER (1 BIT)

PC SRC → **BRANCH & ZERO** → PERMETTONO I **SAITI CONDIZIONATI** MODIFICANDO IL VALORE DEL PC.
↓
SALTO = 1
NON SALTO = 0
↳ 1.0

• **CONTROLLORE ALU** → RICEVE **2 BIT IN INGRESSO** DALL'UNITÀ DI CONTROLLO ($2^2 = 4$ OPERAZIONI DELLA ALU) E **FUNZ3** E **FUNZ7**.

BIT DI CONTROLLO ALU.

0000	→	AND
0001	→	OR
0010	→	ADD *
0110	→	SUB

IN SOSTANZA:

L'**UNITÀ DI CONTROLLO** SI OCCUPA DI GUARDARE L'**OPCODE DELL'ISTRUZIONE**. (7 BIT)
GENERA SEGNALI DI CONTROLLO PER LA **GESTIONE HARDWARE** (COME **MEMWRITE**, **MEMREAD**...) E PER I CALCOLI SI LIMITA A DARE UN ORDINE **GENERICO DI 2 BIT** CHIAMATO **ALUOP**.

IL **CONTROLLORE ALU** È COME UN **ASSISTENTE**, PRENDE IN INGRESSO **ALUOP** E GUARDA I DETTAGLI SU **FUNZ3** E **FUNZ7** PER GENERARE I **4 BIT FINALI** CHE DICONO ALL'ALU COSA FARE (*).

ESEMPIO:

1) **ISTRUZIONI SW E LW**:

ALUOP = 00 → PER ENTRAMBE. IN QUESTO CASO VENGONO SUBITO QUALIFICATE IN **SOMMA** (PER TROVARE L'**INDIRIZZO + OFFSET**) SENZA CONTROLLARE **FUNZ7** E **FUNZ3** (IMPOSTE A 'X' OVEARO "IGNORA QUESTI BIT").
(0010)

2) **BEQ** :

ALOP = 01 ANCHE QUESTA ISTRUZIONE NON HA BISOGNO DI **FUNZ7** E **FUNZ3**
QUINDI VIENE SUBITO CODIFICATA IN **SUB** (0110).

3) **ISTRUZIONI DI TIPO R** (ADD, SUB, AND, OR)

ALOP = 10 → INDICA SOLO CHE È UN TIPO R, QUINDI DOVRÀ COMPLETARE
I VALORI DI **FUNZ7** E **FUNZ3**.

ES: **ALOP** = 10 **FUNZ7** = 0000000 **FUNZ3** = 000 → 0010

ALOP = 10 **FUNZ7** = 0100000 **FUNZ3** = 000 → 0110

E COSÌ VIA PER **OR** E **AND**.

COSA RESTITUISCE L'UNITÀ DI CONTROLLO PER OGNI ISTRUZIONE?

	REGWRITE	ALOP	ALUSRC	BRANCH	MEMREAD	MEMWRITE	MEMTOREG
LOAD	1	00	1	0	1	0	1
STORE	0	00	1	0	0	1	0
BEQ	0	01	0	1	0	0	0
ADD	1	10	0	0	0	0	0
SUB	1	10	0	0	0	0	0
AND	1	10	0	0	0	0	0
OR	1	10	0	0	0	0	0

COME LO LEGGIAMO?

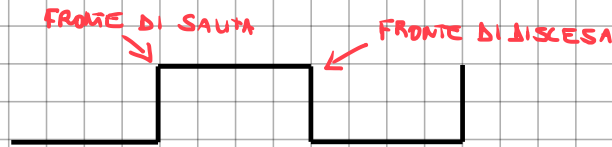
PROVIAMO CON **LOAD** :

- 1) DEVO SCRIVERE IN UN REGISTRO? SÌ → **REGWRITE** = 1
- 2) DEVO USARE UN REGISTRO O UN IMMENATO? IMMENATO → **ALUSRC** = 1
- 3) DEVO FARE UN SALTO? NO → **BRANCH** = 0.
- 4) DEVO LEGGERE DALLA MEMORIA? SÌ, È UNA **LOAD** → **MEMREAD** = 1
- 5) DEVO SCRIVERE IN MEMORIA? NO, NON È UNA **STORE** → **MEMWRITE** = 0
- 6) SCRIVO NEL REGISTRO IL DATO LETTO DALLA MEMORIA O IL RISULTATO DELLA ALU?
IL DATO LETTO IN MEMORIA → **MEMTOREG** = 1

È IMPORTANTE CHE LETTURA E SCRITTURA AVENGANO AL MOMENTO GIUSTO.

• **CLOCK**: SEGNALE PERIODICO E CONTINUO CHE CICLICAMENTE ALTERNA:

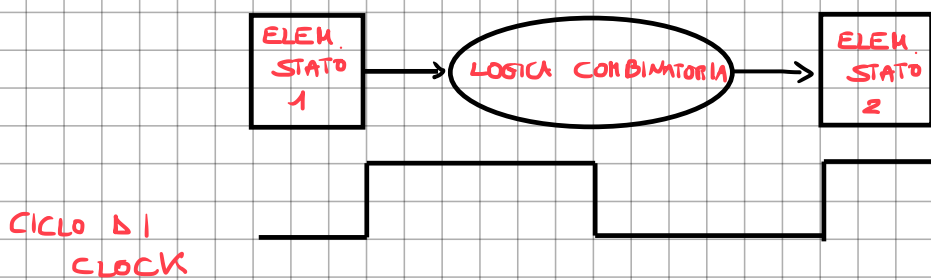
- ALTO
- BASSO



DEFINISCE UNA TEMPORIZZAZIONE COMUNE A TUTTE LE COMPONENTI LOGICHE.

LA SINCRONIZZAZIONE È IMPORTANTE PER EVITARE CHE UN DATO VENGA LETTO E SCRITTO TEMPORANEAMENTE.

IN UN CICLO DI CLOCK, I SEGNALE DEVONO PROPAGARSI DALL'ELEMENTO DI STATO ATTRAVERSO LA LOGICA COMBINATORIA PER RAGGIUNGERE L'ALTRO ELEM. DI STATO.



SE L'ELEMENTO DI STATO:

- VIENE AGGIORNATO AD OGNI CICLO DI CLOCK → NESSUN SEGNALE DI CONTROLLO
- NON VIENE AGGIORNATO AD OGNI CICLO DI CLOCK → SEGNALE DI CONTROLLO

NON AVVIENE ALCUN TIPO DI RETROAZIONE PERCHÉ L'ELEMENTO DI STATO È SOLO EDGE-TRIGGERED.

NEL DATAPATH:

- **FRONTE DI SALITA**: IN QUESTA FASE SI AGGIORNANO GLI ELEMENTI DI STATO:
 - $PC + 4$ / $PC + \text{OFFSET}$. - SE **REGWRITE** = 1, IL REGISTRO VIENE AGGIORNATO.
 - SE **MEMWRITE** = 1, INVIA IL SEGNALE DI SCRITTURA AL CONTROLLER DELLA MEMORIA.
 - SE **MEMREAD** = 1, INVIA IL SEGNALE DI LETTURA AL CONTROLLER DELLA MEMORIA.
- **FINE DEL FRONTE DI SALITA**: PC E REGISTRI HANNO VALORI STABILI E IL DATAPATH INIZIA LA PROPAGAZIONE.
- **FASE ALTA**: **FETCH + DECODE**
- **FRONTE DI DISCESA**: VIENE RECUPERATO IL VALORE DEI REGISTRI E ESTESO IL SEGNO DEL CAMPO IMMEDIATO.
- **FASE BASSA**: **EXECUTE** → SI PROPAGANO I 7 SEGNALE DI CONTROLLO, ALU ESEGUE LE OP., RISOLVE INDIRIZZI, CONFRONTO PER IL BRANCH, $PC + \text{OFFSET}$.
- **RIPARTE DAL FRONTE DI SALITA**.